Assignment: Build a Clone of the Split wise App

Context

You are expected to act as both Product Manager and Developer.

Your assignment is to study Splitwise, reverse engineer its core product behaviour and build a working deployed app using AI.

You will be tested on the technical aspects of your submission. This role is not for a prompt engineer but a full-fledged software engineering intern.

Goal

Build and deploy a simplified Splitwise-inspired app in **2 days**.

You must use an AI tool as your primary development collaborator.

The AI tool should behave like a junior engineer that does not assume requirements. It should ask you detailed product and engineering questions before building.

Minimum product requirements:

1. Login module
2. Create and manage groups. (invite users, add users and remove users)
3. Create and manage expenses
    a. Split equally, unequally, by percentage and by share.
    b. User chat in an expense (real-time updates)
    c. Group wise balances and individual balance summary
    d. Settle debts or record payments
4. Use relational DBs only

Required Deliverables

1. Public deployed app URL
2. GitHub repository
3. README.md with setup instructions and the AI used

4. BUILD_PLAN.md
5. AI_CONTEXT.md
6. Any key prompts used

Core Requirement: AI_CONTEXT.md

This is one of the most important parts of the assignment.

You must create and maintain a file called:

`AI_CONTEXT.md`

This file should contain the full working context used to generate the app.

It should include:

- product understanding
- product scope
- implementation decisions
- engineering requirements
- tech stack
- database schema
- API design
- frontend structure
- deployment plan
- testing plan
- trade-offs
- prompts and AI responses
- changes made during implementation
- known limitations

The AI should continuously update this file as the project evolves.

Important Evaluation Note

We will quiz you on your understanding of the codebase and may ask you to modify certain features.

We might use your AI_CONTEXT.md to try to recreate your app using the same AI tool or a similar one.

Your context file should be detailed enough that another developer or AI agent can rebuild the same app and arrive at a similar codebase.

We will compare:

- your submitted code
- your deployed app

- your AI_CONTEXT.md
- the app recreated from your instructions

If the recreated app differs significantly, it may indicate that the context was incomplete, vague, or not representative of the actual build process.

Required Initial Prompt

Start your project by pasting the following prompt into your AI tool:

```
"You are a junior engineer helping me complete an internship assignment.

The assignment is to reverse engineer Splitwise, scope a realistic 3-day version, and build a working deployed app.

Important instructions:

1. Do not assume product requirements.
2. Do not jump directly into implementation.
3. Ask me detailed questions about product scope, UX, workflows, edge cases, and engineering decisions.
4. Ask about every implementation detail needed to build the app.
5. After each answer I give, update a Markdown file called AI_CONTEXT.md.
6. AI_CONTEXT.md must become the source of truth for the entire project.
7. The final app must be buildable from AI_CONTEXT.md.
8. Another evaluator should be able to paste AI_CONTEXT.md into the same AI tool and recreate a similar app.
9. Before writing code, produce a build plan based only on the agreed context.
10. During implementation, keep updating AI_CONTEXT.md whenever requirements, architecture, schema, UI, or logic changes.
11. Do not recommend technical solutions. Your job is to let me think through the technical solution.

Start by interviewing me.

Ask questions across:
- product goals
- Splitwise research
- core workflows
- user personas
- MVP scope
- out-of-scope features
- data model !IMPORTANT!
- authentication
- groups
- expenses
- settlements
```

```
- balance calculation
- UI screens
- routing
- frontend architecture
- backend architecture
- database choice
- API design
- deployment
- testing
- known risks
- tradeoffs

Do not give me a final plan until you have asked enough questions."
```

BUILD_PLAN.md

Your BUILD_PLAN.md should summarize:

1. Product Research

- How you studied Splitwise
- What you learned
- What workflows you identified
- What product assumptions you made

2. Architecture

- Tech stack
- Database schema
- API design
- Frontend structure
- Deployment approach

3. AI Collaboration Process

- How you instructed the AI
- What questions the AI asked
- How you answered
- How the plan evolved
- How AI_CONTEXT.md was maintained

4. Tradeoffs

- What you simplified
- What you hardcoded
- What you avoided

- What you would improve with more time

Final Note

We are not evaluating whether you perfectly clone Splitwise.

We are evaluating whether you can:

- Understand a real product
- Cater to the requirements mentioned
- Direct an AI agent effectively
- Preserve context clearly
- Build and deploy a working app,
- Create instructions detailed enough to reproduce your work.